REPORT

Reviewer report: pytoolz/toolz#664: Honor scalar pluck

defaults when indexing raises TypeError

Run toolz-664, tested 2026-11-14 12:52 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-14 12:58 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Honor scalar pluck defaults when indexing raises TypeError (pytoolz/toolz#664).

• What we observed: could not verify: claim(s) C1 expect a difference but no observation matches (2 test(s) fail

on main and pass on the PR; 80 test(s) ran on main, 82 on the PR; 2 new test(s) pass, 78 unchanged; probe

model-probe-c1 is not_adequate (the probe crashed or printed no parsed output on every main-side run; not

counted as ran or adequate); its matching results do not count; probe model-probe-c2 is not_adequate (the

probe crashed or printed no parsed output on every main-side run; not counted as ran or adequate); its matching

results do not count; probe model-probe-c3 is not_adequate (the probe crashed or printed no parsed output on

every main-side run; not counted as ran or adequate); its matching results do not count; probe model-probe-c4 is

not_adequate (the probe crashed or printed no parsed output on every main-side run; not counted as ran or

adequate); its matching results do not count).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 10.3 GB (of 15.6 GB).

• Setup check: free disk 92.4 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: toolz python setup install instructions https://github.com/pytoolz/toolz site:github.com/pytoolz/toolz

• Query: toolz python documentation install https://github.com/pytoolz/toolz site:github.com/pytoolz/toolz

• Query: toolz python install requirements https://pypi.org/project/toolz/ https://github.com/pytoolz/toolz

site:pypi.org

• Source: toolz/README.rst at master · pytoolz/toolz · GitHub —

https://github.com/pytoolz/toolz/blob/master/README.rst

• Source: toolz/README.rst at master · pytoolz/toolz · GitHub —

http://github.com/pytoolz/toolz/blob/master/README.rst

• Source: Toolz — https://github.com/pytoolz/toolz/raw/refs/heads/master/README.rst

• Source: toolz/README.rst at 451af60dec590a6010e2babdbf391ea8f815122f · pytoolz/toolz · GitHub —

https://github.com/pytoolz/toolz/blob/451af60dec590a6010e2babdbf391ea8f815122f/README.rst

• Source: toolz — https://pypi.org/project/toolz/

• Setup plan: the same packages on main and pr.

• Setup plan: install coverage (listed in the test extra) on main and pr.

• Setup plan: install pycodestyle (listed in the test extra) on main and pr.

• Setup plan: install pytest (listed in the test extra) on main and pr.

• Setup plan: install sphinx (listed in requirements.txt) on main and pr.

• Setup plan: install furo (listed in requirements.txt) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9

• Installed coverage 7.17.3 from pyproject (listed in the test extra) on main and pr.

• Installed pycodestyle 2.15.0 from pyproject (listed in the test extra) on main and pr.

• Installed pytest 9.2.0 from pyproject (listed in the test extra) on main and pr.

• Installed sphinx 9.1.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed furo 2025.12.19 from requirements.txt (listed in requirements.txt) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (78 unchanged, 3 runs per side) | pytest-pr-1.log:8 «82 passed in 0.95s»

| P-C1 | When scalar pluck is called with a default argument and an invalid index type that would raise TypeError,

the function returns the default value instead of raising TypeError. (PR description) | not testable | probe

model-probe-c1 has no observation for this claim | no matching log line

| P-C2 | Multi-key lookups with invalid index types still raise TypeError. (PR description) | not testable | probe

model-probe-c2 has no observation for this claim | no matching log line

| P-C3 | When no default is provided to scalar pluck, the behavior (including error raising) is unchanged. (PR

description) | not testable | probe model-probe-c3 has no observation for this claim | no matching log line

| P-C4 | Laziness of pluck is unchanged. (PR description) | not testable | probe model-probe-c4 has no

observation for this claim | no matching log line

Citation check (WARN): 1 citation(s) verified; 4 statement(s) have no matching log line.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 80 / 82 | | Tests unchanged | 78 | -

| New tests that pass | 2 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 2 |

toolz.tests.test_itertoolz::test_pluck_default_for_invalid_index_types,

toolz.tests.test_itertoolz::test_pluck_default_keeps_input_lazy

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe-c1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c1.py, frozen

6715d915d49c at 2026-11-14T07:25:27.725164+00:00): Test scalar pluck with default and invalid index type

returns default.

• P-C1 (PR description): When scalar pluck is called with a default argument and an invalid index type that would

raise TypeError, the function returns the default value instead of raising TypeError. Expects the covered

observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-probe-c1-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-probe-c1-pr-1.jsonl:1

«"observation": "probe.error"»

model-probe-c2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c2.py, frozen

a4abad549db3 at 2026-11-14T07:25:27.725295+00:00): Test multi-key pluck with invalid index type still raises

TypeError.

• P-C2 (PR description): Multi-key lookups with invalid index types still raise TypeError. Expects the covered

observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-probe-c2-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-probe-c2-pr-1.jsonl:1

«"observation": "probe.error"»

model-probe-c3 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c3.py, frozen

d9a0ba1300bb at 2026-11-14T07:25:27.725369+00:00): Test scalar pluck without default retains original error

behavior.

• P-C3 (PR description): When no default is provided to scalar pluck, the behavior (including error raising) is

unchanged. Expects the covered observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-probe-c3-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-probe-c3-pr-1.jsonl:1

«"observation": "probe.error"»

model-probe-c4 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c4.py, frozen

bd64d69fbe18 at 2026-11-14T07:25:27.725437+00:00): Test laziness of pluck is unchanged.

• P-C4 (PR description): Laziness of pluck is unchanged. Expects the covered observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 0} |

probe-model-probe-c4-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-probe-c4-pr-1.jsonl:1

«"observation": "probe.error"»

diff-api (diff (auto-derived from base-side code, model-free); informational under baseline full,

diff-probes/diff-api.py, frozen 34c23ecbacf2 at 2026-11-14T07:25:27.728005+00:00): Public functions touched

by the PR, called with type-appropriate inputs on both sides.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| toolz.itertoolz.getter.input0 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(1)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:1 «"observation":

"toolz.itertoolz.getter.input0"»
probe-diff-api-pr-1.jsonl:1 «"observation": "toolz.itertoolz.getter.input0"»

| toolz.itertoolz.getter.input1 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(0)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:2 «"observation":

"toolz.itertoolz.getter.input1"»
probe-diff-api-pr-1.jsonl:2 «"observation": "toolz.itertoolz.getter.input1"»

| toolz.itertoolz.getter.input10 | - | same on both sides | same on both sides: {"repr": "<function

getter.<locals>.<lambda>>", "type": "function"} | probe-diff-api-main-1.jsonl:3 «"observation":

"toolz.itertoolz.getter.input10"»
probe-diff-api-pr-1.jsonl:3 «"observation": "toolz.itertoolz.getter.input10"»

| toolz.itertoolz.getter.input11 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter({})",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:4 «"observation":

"toolz.itertoolz.getter.input11"»
probe-diff-api-pr-1.jsonl:4 «"observation": "toolz.itertoolz.getter.input11"»

| toolz.itertoolz.getter.input2 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(-1)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:5 «"observation":

"toolz.itertoolz.getter.input2"»
probe-diff-api-pr-1.jsonl:5 «"observation": "toolz.itertoolz.getter.input2"»

| toolz.itertoolz.getter.input3 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(2.5)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:6 «"observation":

"toolz.itertoolz.getter.input3"»
probe-diff-api-pr-1.jsonl:6 «"observation": "toolz.itertoolz.getter.input3"»

| toolz.itertoolz.getter.input4 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter('abc')",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:7 «"observation":

"toolz.itertoolz.getter.input4"»
probe-diff-api-pr-1.jsonl:7 «"observation": "toolz.itertoolz.getter.input4"»

| toolz.itertoolz.getter.input5 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(1, 2, 3)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:8 «"observation":

"toolz.itertoolz.getter.input5"»
probe-diff-api-pr-1.jsonl:8 «"observation": "toolz.itertoolz.getter.input5"»

| toolz.itertoolz.getter.input6 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(None)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:9 «"observation":

"toolz.itertoolz.getter.input6"»
probe-diff-api-pr-1.jsonl:9 «"observation": "toolz.itertoolz.getter.input6"»

| toolz.itertoolz.getter.input7 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(True)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:10 «"observation":

"toolz.itertoolz.getter.input7"»
probe-diff-api-pr-1.jsonl:10 «"observation": "toolz.itertoolz.getter.input7"»

| toolz.itertoolz.getter.input8 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(False)",

"type": "itemgetter"} | probe-diff-api-main-1.jsonl:11 «"observation":

"toolz.itertoolz.getter.input8"»
probe-diff-api-pr-1.jsonl:11 «"observation": "toolz.itertoolz.getter.input8"»

| toolz.itertoolz.getter.input9 | - | same on both sides | same on both sides: {"repr": "operator.itemgetter(')", "type":

"itemgetter"} | probe-diff-api-main-1.jsonl:12 «"observation":

"toolz.itertoolz.getter.input9"»
probe-diff-api-pr-1.jsonl:12 «"observation": "toolz.itertoolz.getter.input9"»

| toolz.itertoolz.pluck.input0 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:13 «"observation": "toolz.itertoolz.pluck.input0"»
probe-diff-api-pr-1.jsonl:13

«"observation": "toolz.itertoolz.pluck.input0"»

| toolz.itertoolz.pluck.input1 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:14 «"observation": "toolz.itertoolz.pluck.input1"»
probe-diff-api-pr-1.jsonl:14

«"observation": "toolz.itertoolz.pluck.input1"»

| toolz.itertoolz.pluck.input10 | - | same on both sides | same on both sides: {"repr": "<map object>", "type":

"map"} | probe-diff-api-main-1.jsonl:15 «"observation":

"toolz.itertoolz.pluck.input10"»
probe-diff-api-pr-1.jsonl:15 «"observation": "toolz.itertoolz.pluck.input10"»

| toolz.itertoolz.pluck.input11 | - | same on both sides | same on both sides: {"repr": "<map object>", "type":

"map"} | probe-diff-api-main-1.jsonl:16 «"observation":

"toolz.itertoolz.pluck.input11"»
probe-diff-api-pr-1.jsonl:16 «"observation": "toolz.itertoolz.pluck.input11"»

| toolz.itertoolz.pluck.input2 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:17 «"observation": "toolz.itertoolz.pluck.input2"»
probe-diff-api-pr-1.jsonl:17

«"observation": "toolz.itertoolz.pluck.input2"»

| toolz.itertoolz.pluck.input3 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:18 «"observation": "toolz.itertoolz.pluck.input3"»
probe-diff-api-pr-1.jsonl:18

«"observation": "toolz.itertoolz.pluck.input3"»

| toolz.itertoolz.pluck.input4 | - | same on both sides | same on both sides: {"repr": "<map object>", "type": "map"}

| probe-diff-api-main-1.jsonl:19 «"observation": "toolz.itertoolz.pluck.input4"»
probe-diff-api-pr-1.jsonl:19

«"observation": "toolz.itertoolz.pluck.input4"»

| toolz.itertoolz.pluck.input5 | - | same on both sides | same on both sides: {"repr": "<map object>", "type": "map"}

| probe-diff-api-main-1.jsonl:20 «"observation": "toolz.itertoolz.pluck.input5"»
probe-diff-api-pr-1.jsonl:20

«"observation": "toolz.itertoolz.pluck.input5"»

| toolz.itertoolz.pluck.input6 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:21 «"observation": "toolz.itertoolz.pluck.input6"»
probe-diff-api-pr-1.jsonl:21

«"observation": "toolz.itertoolz.pluck.input6"»

| toolz.itertoolz.pluck.input7 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:22 «"observation": "toolz.itertoolz.pluck.input7"»
probe-diff-api-pr-1.jsonl:22

«"observation": "toolz.itertoolz.pluck.input7"»

| toolz.itertoolz.pluck.input8 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:23 «"observation": "toolz.itertoolz.pluck.input8"»
probe-diff-api-pr-1.jsonl:23

«"observation": "toolz.itertoolz.pluck.input8"»

| toolz.itertoolz.pluck.input9 | - | same on both sides | same on both sides: {"repr": "<map object>", "type": "map"}

| probe-diff-api-main-1.jsonl:24 «"observation": "toolz.itertoolz.pluck.input9"»
probe-diff-api-pr-1.jsonl:24

«"observation": "toolz.itertoolz.pluck.input9"»

Citation check (PASS): 56 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (2.3 s run).

Coverage gate (python, confidence normal): 3 of 3 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| toolz/itertoolz.py | 3 | 3

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested efe5d92644bc of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 3 of 3 changed executable line(s) ran (100%).

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every probe observation cites its output line: 56 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | could not verify: claim(s) C1 expect a difference but no observation

matches (2 test(s) fail on main and pass on the PR; 80 test(s) ran on main, 82 on the PR; 2 new test(s) pass, 78

unchanged; probe model-probe-c1 is not_adequate (the probe crashed or printed no parsed output on every

main-side run; not counted as ran or adequate); its matching results do not count; probe model-probe-c2 is

not_adequate (the probe crashed or printed no parsed output on every main-side run; not counted as ran or

adequate); its matching results do not count; probe model-probe-c3 is not_adequate (the probe crashed or

printed no parsed output on every main-side run; not counted as ran or adequate); its matching results do not

count; probe model-probe-c4 is not_adequate (the probe crashed or printed no parsed output on every main-side

run; not counted as ran or adequate); its matching results do not count) | Read the setup/build log under

Evidence links; build failures usually need a compiler or a newer nightly.

| Probes ran on both sides | WARN | model-probe-c1 main#1; model-probe-c1 pr#1; model-probe-c1 main#2 |

Read the probe-*.log files; a probe that cannot run is not evidence either way.

| Every claim cites a real log line | WARN | 1 citation(s) verified; 4 statement(s) have no matching log line |

Regenerate the report; if it persists, the logs were edited or truncated.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 3 of 3 changed executable line(s) ran (100%)

| Probes ran on both sides | WARN | model-probe-c1 main#1; model-probe-c1 pr#1; model-probe-c1 main#2

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | WARN | 1 citation(s) verified; 4 statement(s) have no matching log line

| Every probe observation cites its output line | PASS | 56 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-14/toolz-664 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/pytoolz/toolz/pull/664

Evidence links

• Pull request pytoolz/toolz#664

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-probe-c1-main-1.jsonl

• probe-model-probe-c1-pr-1.jsonl

• probe-model-probe-c2-main-1.jsonl

• probe-model-probe-c2-pr-1.jsonl

• probe-model-probe-c3-main-1.jsonl

• probe-model-probe-c3-pr-1.jsonl

• probe-model-probe-c4-main-1.jsonl

• probe-model-probe-c4-pr-1.jsonl

• probe-diff-api-main-1.jsonl

• probe-diff-api-pr-1.jsonl

• main run 1 result: pytest-main-1.log:8 «78 passed in 0.87s»

• pr run 1 result: pytest-pr-1.log:8 «82 passed in 0.95s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-14 12:58 IST